

Instructor's notes on Recurrent Neural Networks

Michele Rossi

Department of Information Engineering, University of Padova

6/B, Via G. Gradenigo, I-35131 Padova (PD), Italy

E-mail: {rossi@dei.unipd.it}

Abstract

A few notes detailing how Recurrent Neural Networks are structured, why they are noticeable, how they are trained, via the so called *backpropagation through time* algorithm.

I. RECURRENT NEURAL NETWORKS

As opposed to Feed Forward Neural Networks (FFNN), Recurrent Neural Networks (RNN) are able to model correlation in a data sequence. This is achieved by creating self-transitions inside the network, by which past information (hidden state vector) is propagated through time and used to compute future hidden states and output. A simple but instructive RNN is shown in Fig. 1. The discussion and the derivations that follow consider this diagram, showing how the network transforms an input sequence onto a corresponding output sequence, and how the network parameters (weights) can be trained through *backpropagation*.

Time is discrete $t = 1, 2, \dots$ and is not necessarily connected with the duration of the events, but rather on the sequential order by which they are inputted into the RNN. The input at time t is a vector $\mathbf{x}^{(t)}$ of fixed dimension $\dim(\mathbf{x}^{(t)}) = K$, with $\mathbf{x}^{(t)} = [x_1^{(t)} x_2^{(t)} \dots x_K^{(t)}]^T$. At time t , an internal state $\mathbf{h}^{(t)}$, with size $\dim(\mathbf{h}^{(t)}) = H$, is updated from the new input $\mathbf{x}^{(t)}$ and the previous value of $\mathbf{h}^{(t-1)}$, i.e., the network takes past states (inputs) into account for the computation at the current time step. An output vector $\mathbf{o}^{(t)}$, of size $\dim(\mathbf{o}^{(t)}) = K$, is computed solely based on the internal state $\mathbf{h}^{(t)}$. Since, for now, we consider a *supervised learning* task, at each time t a label (or target output vector) $\mathbf{y}^{(t)}$, with $\dim(\mathbf{y}^{(t)}) = K$, is provided and a *loss* (or error) function $L(t)$ is obtained from $\mathbf{x}^{(t)}$ and $\mathbf{o}^{(t)}$.

Specifically, the equations governing this process are:

$$\mathbf{a}^{(t)} = \mathbf{U}\mathbf{x}^{(t)} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{b} \quad (1)$$

$$\mathbf{h}^{(t)} = \tanh(\mathbf{a}^{(t)}) \quad (2)$$

$$\mathbf{o}^{(t)} = \mathbf{V}\mathbf{h}^{(t)} + \mathbf{c} \quad (3)$$

$$\hat{\mathbf{y}}^{(t)} = \text{softmax}(\mathbf{o}^{(t)}), \quad (4)$$

where the network parameters to be learned are $\boldsymbol{\theta} = \{\mathbf{U}, \mathbf{W}, \mathbf{V}, \mathbf{b}, \mathbf{c}\}$. $\mathbf{a}^{(t)}$, with $\dim(\mathbf{a}^{(t)}) = H$, represents the vector of activations at time t , which is fed to a hyperbolic tangent nonlinear (activation) function, which is applied in an element-wise fashion, namely, $h_i^{(t)} = \tanh(a_i^{(t)})$. The size of the weight matrices $\mathbf{U}, \mathbf{W}, \mathbf{V}$ and biases $\mathbf{b}, \mathbf{c}$ are: $\dim(\mathbf{U}) = H \times K$, $\dim(\mathbf{W}) = H \times H$, $\dim(\mathbf{V}) = K \times H$, $\dim(\mathbf{b}) = H$, and $\dim(\mathbf{c}) = K$, respectively. To avoid any confusion, the softmax layer is also applied

element-wise as follows:

$$\hat{y}_j^{(t)} = \text{softmax}(o_j^{(t)}) = \frac{\exp(o_j^{(t)})}{\sum_{k=1}^K \exp(o_k^{(t)})}, j = 1, \dots, K. \quad (5)$$

Moreover, note that the softmax operator is used as in the following we analyze RNN parameter learning for a *classification* task. For a *regression* task the softmax is no longer needed and vector $\mathbf{o}^{(t)}$ would be used directly, together with the target output vector $\mathbf{y}^{(t)}$ to compute the loss $L(t)$.

Input vectors: we deal with a classification task, where at the input we feed the network with objects from a discrete and finite set of size K , i.e., K objects. Each input $\mathbf{x}^{(t)}$ is thus a one-hot vector that contains a single element equal to one, whereas all the remaining elements are set to zero. The only non-zero element identifies the object that is inputted to the network at time t . In a language processing application, for instance, this construct can be used to identify the word inputted at time t .

Target vectors: the target vector $\mathbf{y}^{(t)}$ corresponds to a one-hot vector denoting the object (e.g., the word) that the network should learn to output at time t in response to the input sequence $\mathbf{x}^{(t)}, \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)}$. In fact, due to the network's recurrent character, each output depends on the entire input sequence, up to the present time step.

Loss function: for a classification task, we use a cross entropy loss function, that at any time t leads to the loss $L(t)$:

$$L(t) = - \sum_{k=1}^K y_k^{(t)} \log(\hat{y}_k^{(t)}). \quad (6)$$

The **total loss** (or error, the two terms are here used interchangeably) is obtained accumulating $L(t)$ across all time steps, namely:

$$L = \sum_{t=1}^T L(t) = - \sum_{t=1}^T \sum_{k=1}^K y_k^{(t)} \log(\hat{y}_k^{(t)}), \quad (7)$$

where T is the last time step and 1 is the first.

Computation of the error gradient: parameter learning is carried out as in a FFNN, i.e., by computing the gradient of the total error L with respect to each network parameter. Once this gradient is obtained, it is utilized within a selected gradient descent algorithm to tune the network parameters towards the desired task. To efficiently compute the gradient, the RNN is unfolded as shown in Fig. 1

(diagram on the right). The unfolded diagram is an acyclic graph, just as the one that we have in any standard FFNN. This means that a standard backpropagation algorithm also applies to the unfolded RNN representation, and that this allows to efficiently compute the total error for each time t . This way of computing the gradient of an RNN is commonly known as Backpropagation Through Time (BTT). Note also that, for any time t , the input in *all* previous time steps $1, \dots, t-1$ ideally concur in the computation of the current output $\mathbf{o}^{(t)}$, which means that they also influence the total error gradient. However, as t grows large it is impractical to unfold the whole RNN diagram from time 1, as this would take a huge amount of memory and computation resources to backpropagate the gradient. As a solution to this, in practice the diagram is unfolded up to a fixed (and small) number of time steps in the past. This is a compromise solution that leads to some degradation in the performance of the learned architecture, but makes learning its parameters practical.

Chain rule of calculus: we briefly review the chain rule of calculus. It is used to compute derivatives of functions constructed by composing other functions, whose derivatives are known. Backpropagation is an algorithm that iteratively uses this chain rule, making it possible to compute the gradient of complex error functions in a highly efficient fashion. To start with, let $x \in \mathbb{R}$ and let $f(\cdot)$ and $g(\cdot)$ be two functions that map a real number onto another real number. For instance, assume $y = g(x)$ and $z = f(g(x)) = f(y)$. The chain rule of calculus allows one to compute dz/dx as:

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}. \quad (8)$$

This can be generalized to the case where $\mathbf{x} \in \mathbb{R}^m$ and $\mathbf{y} \in \mathbb{R}^n$, whereas $g(\cdot) : \mathbb{R}^m \rightarrow \mathbb{R}^n$ and $f(\cdot) : \mathbb{R}^n \rightarrow \mathbb{R}$, with $\mathbf{y} = g(\mathbf{x})$, $z = f(\mathbf{y})$. If $\mathbf{x} = [x_1 \ x_2 \ \dots \ x_m]^T$ and $\mathbf{y} = [y_1 \ y_2 \ \dots \ y_n]^T$, the gradient of z is obtained as:

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}. \quad (9)$$

In vector notation, (9) is written as:

$$\nabla_{\mathbf{x}} z = \left[\frac{\partial z}{\partial x_1} \ \frac{\partial z}{\partial x_2} \ \dots \ \frac{\partial z}{\partial x_m} \right]^T = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^T \nabla_{\mathbf{y}} z, \quad (10)$$

where $\partial \mathbf{y} / \partial \mathbf{x}$ is the $n \times m$ Jacobian matrix of $g(\cdot)$ and

$$\nabla_{\mathbf{y}} z = \left[\frac{\partial z}{\partial y_1} \quad \frac{\partial z}{\partial y_2} \quad \cdots \quad \frac{\partial z}{\partial y_n} \right]^T. \quad (11)$$

II. PARAMETER LEARNING THROUGH BACKPROPAGATION THROUGH TIME

Next, we detail the backpropagation algorithm for the RNN of Fig. 1.

- 1) The first quantity that we compute is the gradient¹ of the total error L (see equation (7)) with respect to the output vector $\mathbf{o}^{(t)}$ at any time $t = 1, \dots, T$,

$$\nabla_{\mathbf{o}^{(t)}} L \triangleq \left[\frac{\partial L}{\partial o_1^{(t)}} \quad \frac{\partial L}{\partial o_2^{(t)}} \quad \cdots \quad \frac{\partial L}{\partial o_K^{(t)}} \right]^T. \quad (12)$$

Applying the chain rule of calculus the i -th element of such gradient is obtained as:

$$\frac{\partial L}{\partial o_i^{(t)}} = \frac{\partial L}{\partial L(t)} \frac{\partial L(t)}{\partial o_i^{(t)}}. \quad (13)$$

From (7), we see that $\partial L / \partial L(t) = 1$, whereas the second term is obtained as follows:

$$\frac{\partial L(t)}{\partial o_i^{(t)}} = - \sum_{k=1}^K y_k^{(t)} \frac{\partial \log \hat{y}_k^{(t)}}{\partial o_i^{(t)}} = - \sum_{k=1}^K y_k^{(t)} \frac{1}{\hat{y}_k^{(t)}} \frac{\partial \hat{y}_k^{(t)}}{\partial o_i^{(t)}} \quad (14)$$

From (5) we obtain:

$$\frac{\partial \hat{y}_k^{(t)}}{\partial o_i^{(t)}} = \begin{cases} \hat{y}_i^{(t)} (1 - \hat{y}_i^{(t)}) & k = i \\ -\hat{y}_i^{(t)} \hat{y}_k^{(t)} & k \neq i. \end{cases} \quad (15)$$

Now, using (15) back into (14), we get:

$$\frac{\partial L(t)}{\partial o_i^{(t)}} = \underbrace{-y_i^{(t)}(1 - \hat{y}_i^{(t)})}_{k=i} + \sum_{k \neq i} y_k^{(t)} \hat{y}_i^{(t)} = -y_i^{(t)} + y_i^{(t)} \hat{y}_i^{(t)} + \sum_{k \neq i} y_k^{(t)} \hat{y}_i^{(t)} = \quad (16)$$

$$= -y_i^{(t)} + \hat{y}_i^{(t)} \underbrace{\sum_{k=1}^K y_k^{(t)}}_1 = \hat{y}_i^{(t)} - y_i^{(t)}, \quad (17)$$

where $\sum_{k=1}^K y_k^{(t)} = 1$ as $\mathbf{y}^{(t)}$ is a one-hot vector. In vector notation, we have:

$$\boxed{\nabla_{\mathbf{o}^{(t)}} L = \hat{\mathbf{y}}^{(t)} - \mathbf{y}^{(t)}}, \quad (18)$$

¹Expressed as a column vector.

that as noted above holds for all time steps $t = 1, \dots, T$.

- 2) At the final time step T , the inner state $\mathbf{h}^{(T)}$ has $\mathbf{o}^{(T)}$ as its only descendant, the error gradient is defined as:

$$\nabla_{\mathbf{h}^{(T)}} L \triangleq \left[\frac{\partial L}{\partial h_1^{(T)}} \quad \frac{\partial L}{\partial h_2^{(T)}} \quad \dots \quad \frac{\partial L}{\partial h_H^{(T)}} \right]^T. \quad (19)$$

The i -th element of this gradient is:

$$\frac{\partial L}{\partial h_i^{(T)}} = \underbrace{\frac{\partial L}{\partial L(T)}}_1 \frac{\partial L(T)}{\partial h_i^{(T)}} = \frac{\partial L(T)}{\partial h_i^{(T)}} \quad (20)$$

$$= \sum_j \frac{\partial L(T)}{\partial o_j^{(T)}} \frac{\partial o_j^{(T)}}{\partial h_i^{(T)}}. \quad (21)$$

Now, from (10), the above equation (20) can be reexpressed in compact form as:

$$\nabla_{\mathbf{h}^{(T)}} L = \left(\frac{\partial \mathbf{o}^{(T)}}{\partial \mathbf{h}^{(T)}} \right)^T \nabla_{\mathbf{o}^{(T)}} L = (\mathbf{V})^T \nabla_{\mathbf{o}^{(T)}} L, \quad (22)$$

where $(\cdot)^T$ is the transpose operator and the second equality follows from (3).

- 3) It is now time to back propagate the gradients through time, from time T to time 1 (no truncation of the unfolded graph is assumed, so we go all the way to the beginning of time). To this end, assume the current time is $t < T$. The inner state $\mathbf{h}^{(t)}$ has descendants $\mathbf{o}^{(t)}$ and $\mathbf{h}^{(t+1)}$. The gradient of the error L is thus derived as:

$$\frac{\partial L}{\partial h_i^{(t)}} = \sum_j \frac{\partial L}{\partial h_j^{(t+1)}} \frac{\partial h_j^{(t+1)}}{\partial h_i^{(t)}} + \sum_j \frac{\partial L}{\partial o_j^{(t)}} \frac{\partial o_j^{(t)}}{\partial h_i^{(t)}}, \quad (23)$$

that following the same line of reasoning as for (22) can be written in compact form as:

$$\begin{aligned} \nabla_{\mathbf{h}^{(t)}} L &= \left(\frac{\partial \mathbf{h}^{(t+1)}}{\partial \mathbf{h}^{(t)}} \right)^T (\nabla_{\mathbf{h}^{(t+1)}} L) + \left(\frac{\partial \mathbf{o}^{(t)}}{\partial \mathbf{h}^{(t)}} \right)^T (\nabla_{\mathbf{o}^{(t)}} L) = \\ &= \mathbf{W}^T \text{diag} \left(1 - (\mathbf{h}^{(t+1)})^2 \right) (\nabla_{\mathbf{h}^{(t+1)}} L) + \mathbf{V}^T (\nabla_{\mathbf{o}^{(t)}} L), \end{aligned} \quad (24)$$

where $\text{diag} \left(1 - (\mathbf{h}^{(t+1)})^2 \right)$ is an $H \times H$ diagonal matrix with diagonal elements $1 - (h_i^{(t+1)})^2$ (we show this shortly below). From (24), we see that $\nabla_{\mathbf{h}^{(t)}} L$ is *recursively computed* from $\nabla_{\mathbf{h}^{(t+1)}} L$, using (22) as the starting point. $\nabla_{\mathbf{o}^{(t)}} L$ is instead a local quantity that is computed independently for each t according to (18). Before moving on, let us shed some light on the second line of

(24). From (3), it promptly follows that $\partial \mathbf{o}^{(t)} / \partial \mathbf{h}^{(t)} = \mathbf{V}$. The computation of the first term of the second line is slightly more involved. We recall that:

$$\mathbf{h}^{(t)} = [h_1^{(t)} \ h_2^{(t)} \ \dots \ h_H^{(t)}]^T \quad (25)$$

$$h_i^{(t+1)} = \tanh(a_i^{(t+1)}) \quad (26)$$

$$a_i^{(t+1)} = (\mathbf{b} + \mathbf{W}\mathbf{h}^{(t)} + \mathbf{U}\mathbf{x}^{(t+1)})_i \quad (27)$$

Applying the derivative of the hyperbolic tangent, from (26), we get

$$\frac{\partial h_i^{(t+1)}}{\partial a_k^{(t+1)}} = \begin{cases} 1 - (h_i^{(t+1)})^2 & i = k \\ 0 & i \neq k, \end{cases} \quad (28)$$

which is the Jacobian of the hyperbolic tangent associated with hidden unit i at time $t + 1$. We are now ready to compute the Jacobian:

$$\frac{\partial h_i^{(t+1)}}{\partial h_j^{(t)}} = \sum_k \frac{\partial h_i^{(t+1)}}{\partial a_k^{(t+1)}} \frac{\partial a_k^{(t+1)}}{\partial h_j^{(t)}} = (1 - (h_i^{(t+1)})^2) \frac{\partial a_i^{(t+1)}}{\partial h_j^{(t)}} = (1 - (h_i^{(t+1)})^2) w_{ij}, \quad (29)$$

where the second equality follows from (28), whereas the last equality follows inspecting (27).

It is not difficult to see that (29) can be compactly expressed as:

$$\frac{\partial \mathbf{h}^{(t+1)}}{\partial \mathbf{h}^{(t)}} = \text{diag} \left(1 - (\mathbf{h}^{(t+1)})^2 \right) \mathbf{W}, \quad (30)$$

that once transposed explains (24).

- 4) The previously obtained derivatives $\nabla_{\mathbf{o}^{(t)}} L$ and $\nabla_{\mathbf{h}^{(t)}} L$ are utilized to derive the derivatives for the network parameters $\boldsymbol{\theta}$. The gradient of the biases $\mathbf{b}, \mathbf{c}$ is:

$$\nabla_{\mathbf{c}} L = \sum_{t=1}^T \left(\frac{\partial \mathbf{o}^{(t)}}{\partial \mathbf{c}} \right)^T \nabla_{\mathbf{o}^{(t)}} L \stackrel{(a)}{=} \sum_{t=1}^T \nabla_{\mathbf{o}^{(t)}} L \quad (31)$$

$$\nabla_{\mathbf{b}} L = \sum_{t=1}^T \left(\frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{b}} \right)^T \nabla_{\mathbf{h}^{(t)}} L \stackrel{(b)}{=} \sum_{t=1}^T \text{diag} \left(1 - (\mathbf{h}^{(t)})^2 \right) \nabla_{\mathbf{h}^{(t)}} L \quad (32)$$

where (a) follows from (3), i.e., the Jacobian $\partial \mathbf{o}^{(t)} / \partial \mathbf{c}$ is the identity matrix, (b) descends from the same calculations done for $\partial \mathbf{h}^{(t+1)} / \partial \mathbf{h}^{(t)}$, by observing that the derivative $\partial a_i^{(t)} / \partial b_j = 1$ if $i = j$ and is zero otherwise.

Let y be a scalar and $\mathbf{X}$ be a matrix of size $m \times n$. We denote by $\nabla_{\mathbf{X}} y$ the matrix of derivatives of y with respect to the elements of x_{ij} of $\mathbf{X}$, formally:

$$\nabla_{\mathbf{X}} y = \begin{bmatrix} \frac{\partial y}{\partial x_{11}} & \frac{\partial y}{\partial x_{12}} & \cdots & \frac{\partial y}{\partial x_{1n}} \\ \frac{\partial y}{\partial x_{21}} & \frac{\partial y}{\partial x_{22}} & \cdots & \frac{\partial y}{\partial x_{2n}} \\ \vdots & \cdots & \ddots & \vdots \\ \frac{\partial y}{\partial x_{m1}} & \frac{\partial y}{\partial x_{m2}} & \cdots & \frac{\partial y}{\partial x_{mn}} \end{bmatrix}. \quad (33)$$

Armed with this definition, we can compute the derivatives of L with respect to the matrix parameters in $\mathbf{V}$, $\mathbf{U}$ and $\mathbf{W}$. For $\mathbf{V}$ we can write:

$$\nabla_{\mathbf{V}} L \stackrel{(a)}{=} \sum_{t=1}^T \sum_{i=1}^K \left(\frac{\partial L}{\partial o_i^{(t)}} \right) \nabla_{\mathbf{V}} o_i^{(t)} \stackrel{(b)}{=} \sum_{t=1}^T (\nabla_{\mathbf{o}^{(t)}} L) (\mathbf{h}^{(t)})^T, \quad (34)$$

where (a) directly follows from the chain rule of calculus, whereas (2) follows from the structure of $\nabla_{\mathbf{V}} o_i^{(t)}$: a close inspection of (3) reveals that

$$\frac{\partial o_i^{(t)}}{\partial v_{kj}} = \begin{cases} h_j^{(t)} & k = i \\ 0 & k \neq i, \end{cases} \quad (35)$$

which means that $\nabla_{\mathbf{V}} o_i^{(t)}$ is a matrix with row i being equal to vector $(\mathbf{h}^{(t)})^T = [h_1^{(t)} \ h_2^{(t)} \ \dots \ h_H^{(t)}]$, while all other elements are zero. The inner sum over i in (34) amounts to summing up K matrices (each of size $K \times H$), where for each the only non-zero row is row i , i.e., the non-zero elements are disjoint and the final result is:

$$\sum_{i=1}^K \left(\frac{\partial L}{\partial o_i^{(t)}} \right) \nabla_{\mathbf{V}} o_i^{(t)} = \begin{bmatrix} h_1^{(t)} \frac{\partial L}{\partial o_1^{(t)}} & h_2^{(t)} \frac{\partial L}{\partial o_1^{(t)}} & \cdots & h_H^{(t)} \frac{\partial L}{\partial o_1^{(t)}} \\ h_1^{(t)} \frac{\partial L}{\partial o_2^{(t)}} & h_2^{(t)} \frac{\partial L}{\partial o_2^{(t)}} & \cdots & h_H^{(t)} \frac{\partial L}{\partial o_2^{(t)}} \\ \vdots & \cdots & \ddots & \vdots \\ h_1^{(t)} \frac{\partial L}{\partial o_K^{(t)}} & h_2^{(t)} \frac{\partial L}{\partial o_K^{(t)}} & \cdots & h_H^{(t)} \frac{\partial L}{\partial o_K^{(t)}} \end{bmatrix}. \quad (36)$$

This explains (b) in equation (34). For matrix $\mathbf{W}$ the following observation is in order. The gradient operator $\nabla_{\mathbf{W}} f$ that is used in calculus takes into account the contribution of $\mathbf{W}$ to the value of f due to all edges in the unfolded computational graph. However, the equations that we want to implement use a recursive argument, which is the usual practice adopted in

backpropagation. According to this recursive view, we would like to compute the contribution of $\mathbf{W}$ to a single edge at a time of the computational graph. To cope with this, we define dummy variables $\mathbf{W}^{(t)}$ that are copies of $\mathbf{W}$ but with each $\mathbf{W}^{(t)}$ only used at time step t . Hence, $\nabla_{\mathbf{W}^{(t)}}$ denotes the (local) contribution of the weights at time t to the overall gradient. Below, we shall clarify this further.

$$\nabla_{\mathbf{W}} L = \sum_{t=1}^T \sum_{i=1}^H \left(\frac{\partial L}{\partial h_i^{(t)}} \right) \nabla_{\mathbf{W}^{(t)} h_i^{(t)}} \stackrel{(a)}{=} \sum_{t=1}^T \text{diag} \left(1 - (\mathbf{h}^{(t)})^2 \right) (\nabla_{\mathbf{h}^{(t)}} L) (\mathbf{h}^{(t-1)})^T. \quad (37)$$

To explain (a) we take a close look at $\nabla_{\mathbf{W}^{(t)} h_i^{(t)}}$. At time t , we have:

$$h_i^{(t)} = \tanh(a_i^{(t)}) \quad (38)$$

$$a_i^{(t)} = (\mathbf{b} + \mathbf{W}^{(t)} \mathbf{h}^{(t-1)} + \mathbf{U} \mathbf{x}^{(t)})_i. \quad (39)$$

From which it follows that:

$$\frac{\partial a_i^{(t)}}{\partial w_{kj}^{(t)}} = \begin{cases} h_j^{(t-1)} & k = i \\ 0 & k \neq i. \end{cases} \quad (40)$$

Moreover, we can write:

$$\frac{\partial h_i^{(t)}}{\partial w_{kj}^{(t)}} = \frac{\partial h_i^{(t)}}{\partial a_i^{(t)}} \frac{\partial a_i^{(t)}}{\partial w_{kj}^{(t)}} = (1 - (h_i^{(t)})^2) \frac{\partial a_i^{(t)}}{\partial w_{kj}^{(t)}} = \begin{cases} (1 - (h_i^{(t)})^2) h_j^{(t-1)} & k = i \\ 0 & k \neq i, \end{cases} \quad (41)$$

which means that $\nabla_{\mathbf{W}^{(t)} h_i^{(t)}}$ is a matrix where only row i is non-zero and is precisely given by (41). All the elements of this non-zero row are then multiplied by $\partial L / \partial h_i^{(t)}$. This explains (a) in (37). Note also that in (37) the internal state variables appear as $\mathbf{h}^{(t-1)}$. When $t = 1$, $\mathbf{h}^{(0)}$ refers to the initial state of the network.

The parameters of the last matrix $\mathbf{U}$ are obtained in a totally similar manner, leading to:

$$\nabla_{\mathbf{U}} L = \sum_{t=1}^T \sum_{i=1}^H \left(\frac{\partial L}{\partial h_i^{(t)}} \right) \nabla_{\mathbf{U}^{(t)} h_i^{(t)}} = \sum_{t=1}^T \text{diag} \left(1 - (\mathbf{h}^{(t)})^2 \right) (\nabla_{\mathbf{h}^{(t)}} L) (\mathbf{x}^{(t)})^T. \quad (42)$$

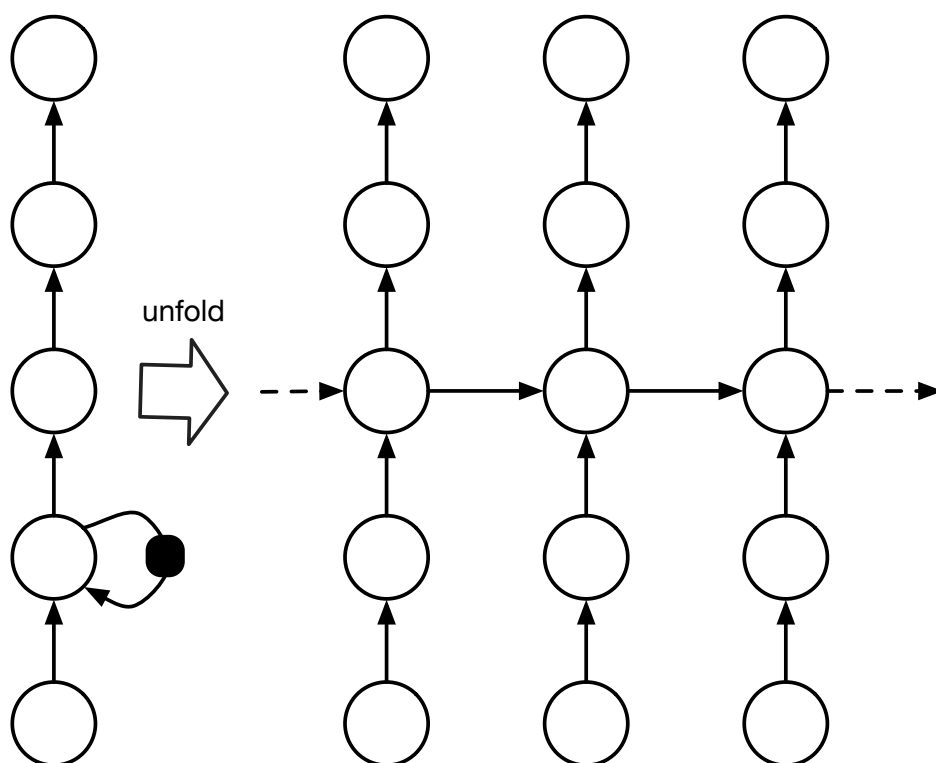


Fig. 1: Computational graph for a baseline RNN. **Left:** compact representation, with a time loop in the internal (inner) state. **Right:** unfolded representation, leading to a feed forward computational graph.